Отчёт по практике 2

Настройка локального CI/CD для FileManagerApp

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Цель работы

Требовалось собрать локальный конвейер CI/CD для приложения из практики 1. Все шаги должны идти в изолированных Docker-контейнерах, чтобы версия .NET на хосте не влияла на результат. В конвейере нужны сборка, тесты с отчётом JUnit, SAST с остановкой на критических находках и локальный деплой. Отдельно проверяются кэш зависимостей, артефакты между стадиями, непривилегированный пользователь в образе и отсутствие секретов в YAML.

2. Стенд

Конвейер рассчитан на GitLab CI и Docker-executor. Файл .gitlab-ci.yml лежит в папке практики, Dockerfile собирает приложение из FileManagerApp. Локально доступны Docker 29.8.1 и .NET SDK 8; сам сервер GitLab с раннером не поднимался, стадии проверены теми же командами, что записаны в конфиге. Это ограничение указано и в README, и в выводах.

Компонент	Значение
Приложение	FileManagerApp, .NET 8, консольное
Базовый образ сборки	mcr.microsoft.com/dotnet/sdk:8.0
Финальный образ	mcr.microsoft.com/dotnet/runtime:8.0 + libsqlite3-0
Размер образа	около 288 МБ
Пользователь	appuser, uid 10001
CI	GitLab CI, четыре стадии

2.1. Соответствие требованиям

Требование задания	Где реализовано	Чем подтверждается
Изолированные Docker-контейнеры	образы sdk/runtime, docker:24.0	build/test/sast/deploy идут в контейнерах
Стадия build	build_app	dotnet restore + publish в Release
Стадия test с JUnit	run_tests	18 testcase, failures=0
Стадия SAST	security_scan	Semgrep, --severity ERROR --error
Стадия local deploy	local_deploy	docker build + docker run --selfcheck

Кэш зависимостей	cache: .nuget/packages	ключ nuget-cache
Артефакты между стадиями	artifacts: publish/	test забирает publish через dependencies
Multi-stage Dockerfile	Dockerfile	SDK остаётся только в builder
Непривилегированный пользователь	USER appuser, uid 10001	запуск образа
Секреты не в коде	CI/CD variables, README	в YAML токенов нет

3. Dockerfile

Dockerfile многоступенчатый. На первом этапе SDK восстанавливает зависимости и публикует приложение в Release. На втором остаётся только runtime, библиотека SQLite и опубликованные файлы; SDK, исходники и кэш NuGet в финальный образ не попадают. Создаётся пользователь appuser с uid 10001, и USER переключает контейнер на него. Это уменьшает последствия ошибки в приложении: процесс не работает от root.

Для SQLite в runtime-образ ставится libsqlite3-0. Приложение запускается командой dotnet FileManagerApp.dll; при старте без аргументов открывается меню, а с --selfcheck выполняются 18 проверок. В контейнере проверка проходит, код возврата 0.

4. Стадии конвейера

Стадия	Что делает	Артефакт / кэш
build	restore и publish в Release	publish/, кэш NuGet
test	запускает --selfcheck и превращает OK в JUnit XML	test-results.xml
sast	Semgrep по исходникам, --severity ERROR --error	логи и находки
deploy	docker build и docker run --selfcheck	локальный образ

Кэш NuGet вынесен в .nuget/packages и привязан ключом nuget-cache, поэтому повторная сборка не скачивает пакеты заново. publish/ передаётся из build в test через dependencies; JUnit-отчёт публикуется в artifacts.reports.junit, чтобы GitLab показывал тесты во вкладке Tests. Semgrep запускается после cd в папку приложения, потому что кириллический путь он не принимает как корень сканирования. У джобы стоит entrypoint: [""]: иначе образ semgrep/semgrep передаёт cd в сам сканер.

Деплой использует docker:24.0 и сервис docker:24.0-dind. Переменные DOCKER_HOST=tcp://docker:2375 и DOCKER_TLS_CERTDIR="" направляют клиент на дочерний демон. Порт не пробрасывается: приложение консольное, HTTP-сервера у него нет. Вместо проверки порта контейнер запускается с --selfcheck и проверяется код возврата; для веб-приложения здесь было бы docker run -d -p 8080:8080.

5. Проверка

Публикация и JUnit:

dotnet publish -c Release -o publish
bash "Практика2_CI_CD/scripts/run_selfcheck_as_junit.sh" publish/FileManagerApp.dll test-results.xml

В XML получилось tests="18" failures="0" errors="0". Скрипт разбирает строки OK: и делает по каждой отдельный <testcase>; если самопроверка падает, он завершает работу с ненулевым кодом.

Сборка и запуск образа:

docker build -t filemanagerapp:local -f Практика2_CI_CD/Dockerfile .
docker run --rm filemanagerapp:local --selfcheck

Образ собрался, размер около 288 МБ, контейнер вернул 0. Semgrep прогнал 52 правила по 11 файлам и не нашёл ошибок уровня ERROR. Полноценный GitLab-раннер не запускался: это единственное, что проверено локально, а не в веб-интерфейсе GitLab.

5.1. Артефакты конвейера

Артефакт у каждой стадии свой, и в GitLab он доступен кнопкой Artifacts у джобы, а JUnit-отчёт — вкладкой Tests у пайплайна. Чтобы показать то же самое без развёрнутого сервера, добавлен скрипт run_jobs_locally.sh: он запускает каждую джобу в том же образе и с теми же командами, что записаны в .gitlab-ci.yml, и раскладывает результат по папкам artifacts/<имя_джобы>. Прогон занимает около двух минут и перезаписывает снимки.

Джоба	Образ	Артефакт	Что в нём видно
build_app	dotnet/sdk:8.0	publish/	FileManagerApp.dll, .deps.json, .runtimeconfig.json, .pdb, launcher
run_tests	dotnet/sdk:8.0	test-results.xml	tests="18" failures="0" errors="0"
security_scan	semgrep/semgrep	job.log	Ran 52 rules on 11 files: 0 findings, код возврата 0
local_deploy	docker:24.0	image.inspect.txt	User=appuser, размер 288 МБ, entrypoint без SDK-слоёв

Именно этот прогон нашёл настоящую ошибку в конфигурации. Стадия run_tests брала образ dotnet/sdk:8.0 и запускала в нём уже собранный бинарник, но пакета libsqlite3 в этом образе нет. На Windows это не проявляется: там winsqlite3.dll входит в состав системы, и загрузчик находит SQLite через неё. В Linux-контейнере загрузчик ищет libsqlite3.so.0, не находит, и первый же выход к базе роняет самопроверку с сообщением о незагруженной библиотеке — в логе это выглядело как tests="1" failures="1".

Исправление: в run_tests перед запуском добавлена установка libsqlite3-0 — того же пакета, который уже стоит в Dockerfile для финального runtime-образа. После этого стадия проходит честно: 18 проверок из 18, код возврата 0, и тот же результат повторяется внутри собранного образа на стадии деплоя.

Что при этом остаётся непроверенным: очередь раннеров, хранение артефактов на сервере GitLab и срок их жизни (expire_in: 1 day). Локально артефакты просто копируются в папку, а не загружаются на сервер.

6. Секреты и риски

В .gitlab-ci.yml нет токенов и паролей. Для приватного registry предусмотрены masked/protected variables CI_REGISTRY_USER и CI_REGISTRY_PASSWORD; локально они задаются переменными окружения. Проброс docker.sock и --docker-privileged допустимы только на учебной машине: они дают джобе большой контроль над хостом. В продакшене нужен отдельный изолированный runner и минимальные права.

7. Ограничения и выводы

Полный сервер GitLab и раннер не разворачивались; стадии проверены локальными командами, идентичными конфигу. Это позволяет проверить сборку, тесты, SAST и образ, но не проверяет поведение самого GitLab (очередь, переменные, UI). Для учебной работы этого достаточно, и ограничение отражено в README.

В итоге конвейер закрывает все пункты задания: изолированные контейнеры, build, тесты с JUnit, SAST с падением на ERROR, кэш NuGet, артефакты между стадиями и локальный деплой. Dockerfile многоступенчатый, приложение работает от непривилегированного пользователя, секретов в конфиге нет.